

This is one of the best ways to master SQL for interviews and real-world projects:

SQL Learning Matrix (Real-World Scenarios)

Level	Concepts
Basic	SELECT, WHERE, ORDER BY, LIMIT
Intermediate	JOINS, GROUP BY, HAVING
Advanced	Subqueries, Derived Tables, CTEs
Expert	Window Functions, Ranking, Running Totals
Real-Time	Date Functions, String Functions, Conditional Logic
Data Warehouse	Analytical Queries, KPI Calculations
Production	Performance, Indexing, Aggregations

DOMAIN 1: EMPLOYEE MANAGEMENT SYSTEM

Tables

employees

```
CREATE TABLE employees
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(100),
  department VARCHAR(50),
  salary DECIMAL(10,2),
  manager_id INT,
  hire_date DATE
);
```

Insert Data

```
INSERT INTO employees VALUES  
(1,'Raj', 'IT', 90000, NULL, '2018-01-10'),  
(2,'Priya','HR',70000,1,'2019-03-15'),  
(3,'Kumar','IT',85000,1,'2020-05-12'),  
(4,'John','Finance',75000,2,'2021-02-01'),  
(5,'Anita','IT',95000,1,'2017-09-20'),  
(6,'Vijay','HR',68000,2,'2022-01-01');
```

1. SELECT + WHERE

Employees earning more than 80K

```
SELECT *  
FROM employees  
WHERE salary > 80000;
```

2. SELECT + ORDER BY

Highest salary employees

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

3. SELECT + STRING FUNCTIONS

Employee names in uppercase

```
SELECT  
emp_name,  
UPPER(emp_name)  
FROM employees;
```

4. SELECT + DATE FUNCTIONS

Years of Service

MySQL

```
SELECT  
emp_name,  
TIMESTAMPDIFF(YEAR,hire_date,CURDATE()) AS years_service  
FROM employees;
```

PostgreSQL

```
SELECT  
emp_name,  
EXTRACT(YEAR FROM AGE(CURRENT_DATE,hire_date))  
FROM employees;
```

5. GROUP BY

Department Wise Salary

```
SELECT  
department,  
SUM(salary) total_salary  
FROM employees  
GROUP BY department;
```

6. GROUP BY + HAVING

Departments spending more than 150K

```
SELECT  
department,  
SUM(salary)  
FROM employees  
GROUP BY department  
HAVING SUM(salary) > 150000;
```

7. SELF JOIN

Employee and Manager

```
SELECT  
e.emp_name employee,  
m.emp_name manager  
FROM employees e  
LEFT JOIN employees m  
ON e.manager_id=m.emp_id;
```

8. SUBQUERY

Employees earning above average

```
SELECT *  
FROM employees  
WHERE salary >  
(  
    SELECT AVG(salary)  
    FROM employees  
);
```

9. CORRELATED SUBQUERY

Highest paid employee in each department

```
SELECT *  
FROM employees e1  
WHERE salary =  
(  
    SELECT MAX(salary)  
    FROM employees e2  
    WHERE e1.department=e2.department  
);
```

10. DERIVED TABLE

```
SELECT *  
FROM
```

```
(  
  SELECT department,  
         AVG(salary) avg_salary  
  FROM employees  
  GROUP BY department  
) d  
WHERE avg_salary > 80000;
```

11. WINDOW FUNCTION

Department Salary Ranking

```
SELECT  
emp_name,  
department,  
salary,  
RANK() OVER  
(  
  PARTITION BY department  
  ORDER BY salary DESC  
) rank_no  
FROM employees;
```

12. DENSE_RANK

```
SELECT  
emp_name,  
salary,  
DENSE_RANK() OVER  
(  
  ORDER BY salary DESC  
) rank_no  
FROM employees;
```

13. ROW_NUMBER

```
SELECT  
ROW_NUMBER() OVER
```

```
(  
ORDER BY salary DESC  
) serial_no,  
emp_name  
FROM employees;
```

14. LAG

```
SELECT  
emp_name,  
salary,  
LAG(salary)  
OVER(ORDER BY salary) previous_salary  
FROM employees;
```

15. LEAD

```
SELECT  
emp_name,  
salary,  
LEAD(salary)  
OVER(ORDER BY salary) next_salary  
FROM employees;
```

16. RUNNING TOTAL

```
SELECT  
emp_name,  
salary,  
SUM(salary)  
OVER  
(  
ORDER BY emp_id  
) running_total  
FROM employees;
```

DOMAIN 2: ORDER MANAGEMENT

Tables

```
CREATE TABLE customers
(
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100),
  city VARCHAR(50)
);
```

```
CREATE TABLE orders
(
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE,
  amount DECIMAL(10,2)
);
```

Data

```
INSERT INTO customers VALUES
(1,'Amazon','Mumbai'),
(2,'Flipkart','Bangalore'),
(3,'Reliance','Chennai');
```

```
INSERT INTO orders VALUES
(101,1,'2025-01-01',1000),
(102,1,'2025-01-02',1500),
(103,2,'2025-01-03',2000),
(104,3,'2025-01-04',2500),
(105,2,'2025-01-05',3000);
```

17. JOIN

```
SELECT
c.customer_name,
o.amount
FROM customers c
JOIN orders o
```

```
ON c.customer_id=o.customer_id;
```

18. GROUP BY + JOIN

```
SELECT  
c.customer_name,  
SUM(o.amount) total_sales  
FROM customers c  
JOIN orders o  
ON c.customer_id=o.customer_id  
GROUP BY c.customer_name;
```

19. TOP CUSTOMER

```
SELECT *  
FROM  
(  
  SELECT  
    customer_id,  
    SUM(amount) total_sales,  
    RANK() OVER  
    (  
      ORDER BY SUM(amount) DESC  
    ) rnk  
  FROM orders  
  GROUP BY customer_id  
) x  
WHERE rnk=1;
```

20. MONTHLY SALES

```
SELECT  
YEAR(order_date),  
MONTH(order_date),  
SUM(amount)  
FROM orders  
GROUP BY  
YEAR(order_date),
```



```
MONTH(order_date);
```

DOMAIN 3: HOSPITAL MANAGEMENT

Tables

```
CREATE TABLE patients
(
  patient_id INT,
  patient_name VARCHAR(100),
  disease VARCHAR(50),
  admission_date DATE,
  bill_amount DECIMAL(10,2)
);
```

Data

```
INSERT INTO patients VALUES
(1,'Ramesh','Fever','2025-01-01',5000),
(2,'Sita','Covid','2025-01-05',15000),
(3,'Kiran','Fever','2025-01-08',6000),
(4,'John','Diabetes','2025-01-10',12000);
```

21. Disease Wise Patients

```
SELECT
disease,
COUNT(*)
FROM patients
GROUP BY disease;
```

22. Most Expensive Treatment

```
SELECT *
FROM patients
WHERE bill_amount=
```

```
(  
SELECT MAX(bill_amount)  
FROM patients  
);
```

23. Disease Revenue Ranking

```
SELECT  
disease,  
SUM(bill_amount),  
RANK() OVER  
(  
ORDER BY SUM(bill_amount) DESC  
)  
FROM patients  
GROUP BY disease;
```

Advanced Combinations (Frequently Asked in Interviews)

SELECT + JOIN + GROUP BY + HAVING

```
SELECT  
c.customer_name,  
SUM(o.amount) total_sales  
FROM customers c  
JOIN orders o  
ON c.customer_id=o.customer_id  
GROUP BY c.customer_name  
HAVING SUM(o.amount)>2000;
```

SELECT + JOIN + SUBQUERY

```
SELECT *  
FROM customers  
WHERE customer_id IN  
(
```

```
SELECT customer_id
FROM orders
WHERE amount > 2000
);
```

SELECT + JOIN + WINDOW FUNCTION

```
SELECT
c.customer_name,
o.amount,
RANK() OVER
(
PARTITION BY c.customer_name
ORDER BY o.amount DESC
)
FROM customers c
JOIN orders o
ON c.customer_id=o.customer_id;
```

SELECT + GROUP BY + HAVING + WINDOW

```
SELECT *
FROM
(
SELECT
department,
SUM(salary) total_salary,
RANK() OVER
(
ORDER BY SUM(salary) DESC
) rnk
FROM employees
GROUP BY department
) x
WHERE total_salary > 100000;
```

Additional Functions You Must Learn

Numeric

ROUND()
CEIL()
FLOOR()
ABS()
MOD()
POWER()
SQRT()

String

UPPER()
LOWER()
TRIM()
SUBSTRING()
REPLACE()
CONCAT()
LENGTH()

Date

NOW()
CURRENT_DATE
DATEDIFF()
DATE_ADD()
DATE_SUB()
EXTRACT()

NULL Handling

COALESCE()
IFNULL()
NULLIF()
NVL()

Conditional

CASE
WHEN
THEN
ELSE
END

Real Interview Combinations Checklist

Practice all these combinations:

SELECT + WHERE

SELECT + ORDER BY

SELECT + GROUP BY

SELECT + GROUP BY + HAVING

SELECT + JOIN

SELECT + JOIN + WHERE

SELECT + JOIN + GROUP BY

SELECT + JOIN + GROUP BY + HAVING

SELECT + SUBQUERY

SELECT + CORRELATED SUBQUERY

SELECT + DERIVED TABLE

SELECT + CTE

SELECT + WINDOW FUNCTION

SELECT + WINDOW + PARTITION BY

SELECT + WINDOW + ORDER BY

SELECT + JOIN + WINDOW

SELECT + GROUP BY + WINDOW

SELECT + DATE FUNCTIONS

SELECT + STRING FUNCTIONS

SELECT + CASE

SELECT + COALESCE

SELECT + LAG

SELECT + LEAD

SELECT + FIRST_VALUE

SELECT + LAST_VALUE

SELECT + NTILE

SELECT + PERCENT_RANK

SELECT + CUME_DIST

SELECT + ROLLUP

SELECT + CUBE

SELECT + GROUPING SETS

If you're preparing trainees or interviews, the next step would be a **100-problem SQL workbook** (easy → medium → hard → expert) using the same Employee, Orders, Hospital, Banking, E-Commerce, Manufacturing, Telecom, and Insurance domains with complete schemas, data, questions, and solutions. That format closely resembles HackerRank, LeetCode SQL, TCS, Infosys, Accenture, Cognizant, Capgemini, and product-company SQL rounds.